

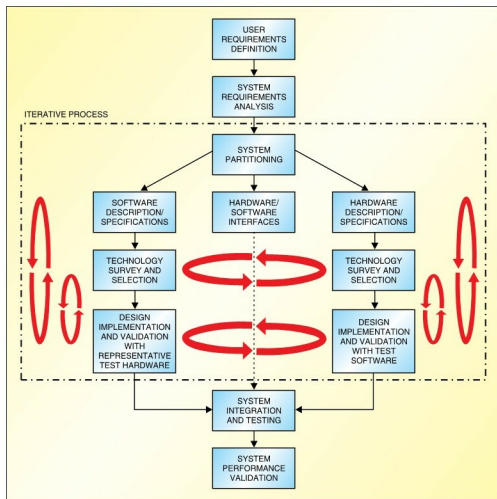


Fig. 2: Modern design flow using hardware-software co-design techniques (Courtesy: Ashish Gupta, The Learning Labs)

in synchronous development of hardware and software. Though partitioning is required to define the functionality, the thought of hardware-software interface begins here itself.

Gupta shares, "The key to a good hardware-software co-design is the system partitioning scheme chosen, or rather the partitioning scheme that emerges out as a result of iterative, rapid prototyping and evaluation of various options. The next important thing to do is to rapidly prototype the design and evaluate the various schemes in terms of the requirement metrics chosen. And this needs to be an iterative process and may require multiple cycles of technology survey and prototyping across hardware and software domains."

Again, at every stage, it is impor-

tant to keep the hardware and software teams synced with each other and ensure information flow and regular updates about changes during the design phase.

Use of simulators

Simulation is a process of verifying the abstract model of the design (described in hardware description languages like VHDL, Verilog and SystemC) on a computer using a software tool called 'simulator.' In the context of embedded systems and co-verification, one of the simulation-based approaches that the designers can take is "to have both the hardware and processor models written at RTL and have the application software written in 'C' or similar language and run the entire system simulation us-

ing a hardware simulation environment," shares Mallikarjuna. However, this approach could be cumbersome because it takes lots of simulation cycles and debugging the system could be cumbersome. He suggests that a better approach is to use an instruction set simulator (ISS) for the processor part together with a hardware simulator for the hardware part to co-verify the system-level environment using tools like Questa Codelink."

Some of the tools like Cadence's Incisive Enterprise Simulator (IES) allow engineers to co-verify hardware and software in the system context, performing both high-speed hardware verification and hardware-software co-verification, thereby simulating system-level behaviour to reduce the risk.

Emulators and hardware-assisted verification

There is no guarantee that your verified RTL will function as desired after tape-out once the embedded software is loaded onto the hardware. Neither can you find an easy solution when you hit a bug after you download the algorithm onto the actual hardware.

An emulator allows software debugging at the RTL stage, which is the earliest stage at which co-verification can happen. Fast emulation has become a critically important verification component for large and complex SoCs. Emulation is today adopted not only to test the hardware aspects of an SoC design but also to verify integration of the hardware and embedded software.

"Logic simulation is excellent in debugging RTL code but it is not up